

FUZZYTUTOR

Master Study Guide

Fuzzy theory, code implementation, and oral-exam preparation

ANFIS mastery engine

Mamdani cognitive-friction engine

Adaptive task-selection controller

Django + React system flow

Prepared from the implemented FuzzyTutor codebase

Study edition - August 2026

HOW TO USE THIS GUIDE

Study map

This guide explains the same system at three levels: the fuzzy-logic theory, the exact Python implementation, and the end-to-end behavior visible in the application. Read Chapters 1-3 for the big picture, Chapters 4-7 for the two engines, and Chapters 8-10 for adaptation, verification, and presentation preparation.

The most important fact to remember is that FuzzyTutor deliberately separates knowledge from friction. ANFIS asks, 'How strong is the learner's current performance evidence?' Mamdani asks, 'How difficult does the interaction appear behaviorally?' The controller considers both before choosing a task.

If you have...	Study these sections
10 minutes	Executive summary, engine comparison, decision thresholds, rapid-fire questions
30 minutes	Add fuzzy foundations, code pipeline, and the worked inference
1-2 hours	Read the complete guide and reproduce the worked calculations
Presentation tomorrow	Memorize the one-minute explanation and rehearse the viva questions

One-sentence project explanation

FuzzyTutor is a Django-and-React adaptive programming tutor that uses a trained first-order TSK/ANFIS engine to estimate mastery and a Mamdani engine with centroid defuzzification to estimate cognitive friction, then selects the nearest suitable non-repeating task while exposing the complete reasoning trace.

Contents

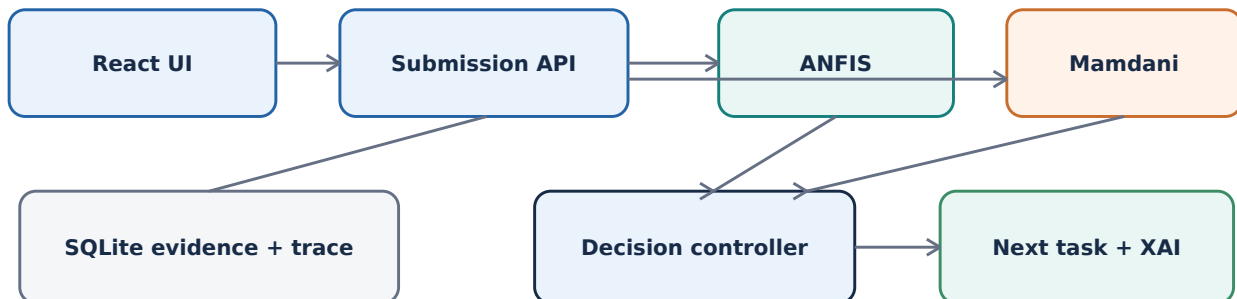
Study map	2
Contents	3
System overview	5
What happens after a submission	5
Why two engines?	5
Technology and code ownership	6
Fuzzy-logic foundations	7
Crisp sets versus fuzzy sets	7
Membership functions used in FuzzyTutor	7
Fuzzy operators	7
The general inference cycle	7
From HTTP request to fuzzy inputs	8
Public submission contract	8
How each input is derived	8
The exact orchestration call	8
Why code tasks are not marked correct	8
ANFIS mastery engine	9
What kind of ANFIS is implemented?	9
Inputs and performance evidence	9
Premise membership sets	9
The five rules	9
The five ANFIS layers	9
ANFIS training and evaluation	10
What is learned and what stays fixed	10
Synthetic bootstrap records	10
Gradient update	10
Stored model metadata	10
Reproduction commands	10
Mamdani cognitive-friction engine	11
Purpose and inputs	11
Output sets	11
Behavioral rules	11
Mamdani inference in code	11
Complete worked inference	12
ANFIS calculation	12
Mamdani calculation	12
Controller result	12
Decision controller and adaptation	13

Focus-state thresholds	13
Difficulty recommendation	13
Selecting the actual task	13
Smoothing over time	13
Module exit policy	13
Explainability, persistence, and UI	15
What engineTrace contains	15
Learner-facing explanation	15
Core persisted models	15
Security and evidence integrity	15
Code map and debugging route	16
Standalone engine demonstration	16
Recommended debugging sequence	16
Verification and limitations	17
What the automated tests establish	17
Important limitations	17
A subtle implementation point	17
Run the verification suite	17
Oral-exam and presentation preparation	18
One-minute explanation	18
Likely viva questions	18
Rapid-fire facts to memorize	19
Compact formula sheet	20
Glossary	20
References and implementation sources	21
Implementation sources	21
Theory references	21

CHAPTER 1

System overview

FuzzyTutor teaches seven programming topics through 105 curated tasks. Each module contains 15 tasks: five foundation, five intermediate, and five advanced. The task bank mixes multiple-choice questions with code-writing activities.



What happens after a submission

- The React interface sends the session token, current task ID, elapsed time, and the learner's answer or explicit skip.
- The Django serializer validates that this really is the current task and derives completion. It rejects client-supplied correctness, completion, hint counts, or answer metadata.
- The submission view calculates relative time, MCQ correctness, and the number of server-recorded hints.
- The ANFIS engine predicts Knowledge Mastery on a 0-100 scale.
- The Mamdani engine predicts System Cognitive Friction on a 0-100 scale.
- The controller converts both outputs into a focus state, recommendation, support message, and target difficulty.
- The task selector chooses the closest unused task, persists the complete trace, and returns the new state to React.

Why two engines?

Question	ANFIS mastery	Mamdani friction
Meaning	What does the evidence suggest about current knowledge?	How much behavioral difficulty is visible?
Main inputs	Task weight, prior mastery, completion, correctness evidence	Relative time, hints, completion, task type
Output method	Normalized weighted first-order TSK consequents	Centroid of clipped and aggregated output sets
Uses correctness?	Yes for MCQs; conservative unknown signal for code	No - a wrong answer is not automatically frustration
Why useful	Estimates academic readiness	Detects delay, assistance, and incomplete progress

A learner may know the material but still need hints, or may answer quickly while being wrong. A single score would hide those combinations. Keeping the dimensions separate is the central design decision.

Technology and code ownership

Area	Role	Primary code
Frontend	Tutor workspace, gauges, XAI panel, surveys	frontend/src/
Learning backend	Sessions, task bank, hints, submissions, progress	backend/apps/learning/
Fuzzy backend	Memberships, ANFIS, Mamdani, controller	backend/apps/fuzzy/
Persistence	SQLite models and complete inference logs	backend/db.sqlite3 and Django models
Deployment	Separate frontend/backend containers	docker-compose.yml

CHAPTER 2

Fuzzy-logic foundations

Crisp sets versus fuzzy sets

A crisp set gives an element either membership 0 or membership 1. A fuzzy set gives a degree between 0 and 1. If response time 1.35 belongs to 'normal' with degree 0.222 and to 'high' with degree 0.211, both descriptions can participate in the decision.

A membership degree is not a probability. Probability describes uncertainty about whether an event occurs. Fuzzy membership describes how well a value matches a linguistic concept.

Membership functions used in FuzzyTutor

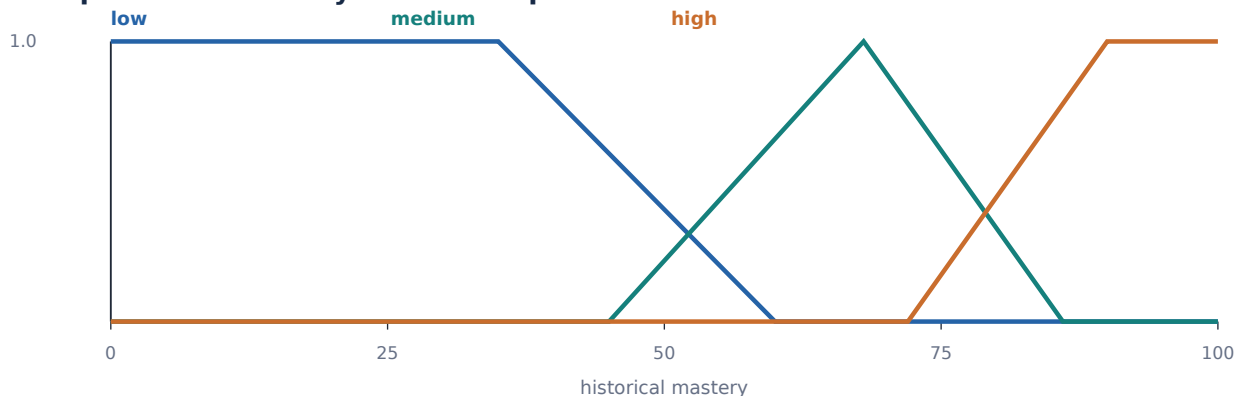
The implementation uses three transparent piecewise-linear shapes.

Triangle(a,b,c): 0 outside [a,c]; $(x-a)/(b-a)$ while $a < x < b$; $(c-x)/(c-b)$ while $b \leq x < c$.

LeftShoulder(a,b): 1 for $x \leq a$; $(b-x)/(b-a)$ for $a < x < b$; 0 for $x \geq b$.

RightShoulder(a,b): 0 for $x \leq a$; $(x-a)/(b-a)$ for $a < x < b$; 1 for $x \geq b$.

Example: ANFIS history memberships



Fuzzy operators

Logical idea	Code operation	Example
A AND B	$\min(A, B)$	$\min(0.8, 0.6) = 0.6$
A OR B	$\max(A, B)$	$\max(0.2, 0.7) = 0.7$
Rule implication	$\min(\text{rule strength, output membership})$	Clip an output shape at the firing strength
Rule aggregation	$\max(\text{all clipped outputs})$	Keep the strongest support at each output point

The general inference cycle

- Fuzzification: turn crisp inputs into linguistic membership degrees.
- Rule evaluation: compute each rule's firing strength using min and max.
- Implication/consequent evaluation: produce each rule's output contribution.
- Aggregation: combine all contributions.
- Defuzzification: convert the aggregate into a crisp number.

CHAPTER 3

From HTTP request to fuzzy inputs

Public submission contract

```
{
  "sessionToken": "...",
  "taskId": "lists-03",
  "elapsedTimeSeconds": 81.4,
  "selectedChoice": "...",      // MCQ
  // or "answerText": "..."    // code task
  // or "skipped": true
}
```

The browser is not trusted to send `isCorrect`, `completionRatio`, `assistanceInteractions`, or private answer metadata. The serializer explicitly rejects these fields.

How each input is derived

Engine input	Source	Runtime meaning
taskMetricWeight	Private task metadata	35 foundation, 55 intermediate, 75 advanced
historicalGradeAverage	ModuleProgress.aggregate_mastery	Smoothed module-local mastery before this attempt
relativeResponseTime	elapsed / baseline	1.0 expected time; 2.0 twice expected time
assistanceInteractions	Stored HintEvent count	0-3 progressive hints actually revealed
completionRatio	Server validation	0 for skip; 1 for valid MCQ or meaningfully edited code
taskType	Private task metadata	mcq or code
isCorrect	Private MCQ answer key	True/False for MCQ; None for code

The exact orchestration call

```
fuzzy_inputs = {
  "taskMetricWeight": task["taskMetricWeight"],
  "historicalGradeAverage": module_progress.aggregate_mastery,
  "relativeResponseTime": elapsed / task["baselineTimeSeconds"],
  "assistanceInteractions": len(revealed_hint_levels),
  "completionRatio": data["completionRatio"],
  "taskType": task["type"],
  "isCorrect": is_correct,
}
fuzzy_result = evaluate_learning_state(fuzzy_inputs)
```

Why code tasks are not marked correct

The project intentionally does not execute arbitrary learner code. A meaningful edit counts as completion, not correctness. This avoids falsely labelling any non-empty program as correct and avoids introducing compiler/test feedback as another friction signal.

The consequence is important: the mastery engine receives weaker evidence from code tasks than from correct MCQs. Module exit therefore also requires recent graded MCQ evidence.

CHAPTER 4

ANFIS mastery engine

What kind of ANFIS is implemented?

FuzzyTutor implements a transparent first-order Takagi-Sugeno-Kang model with ANFIS-style layers. Premise membership functions and the compact pedagogical rule base are fixed. The first-order consequent coefficients are trained and stored in a versioned JSON artifact.

Inputs and performance evidence

Evidence	Value
Correct MCQ	100
Incorrect MCQ	20
Completed, ungraded code	$40 + 25(1.0) = 65$
Skipped/ungraded attempt	$40 + 25(0.0) = 40$

Premise membership sets

Variable	Set	Shape and parameters
Challenge	Foundation / Intermediate / Advanced	left(35,60) / tri(35,58,82) / right(65,85)
History	Low / Medium / High	left(35,60) / tri(45,68,86) / right(72,90)
Completion	Low / Medium / High	left(0.20,0.65) / tri(0.35,0.70,0.98) / right(0.75,1.00)
Performance	Weak / Emerging / Strong	left(35,65) / tri(45,70,90) / right(75,95)

The five rules

Rule	Antecedent
Secure prior mastery	high history AND high completion AND strong performance
Developing mastery	(medium history AND high completion) OR (high history AND emerging performance)
Productive challenge	advanced challenge AND high completion AND strong performance
Fragile progress	(medium history AND medium completion) OR (emerging performance AND medium completion)
Knowledge gap	(low history AND weak performance) OR (low completion AND weak performance)

Coverage guard: if none of the compact named rules fires, the engine visibly routes the mixed state through the developing-mastery consequent. The trace records `coverageGuardUsed=true`.

The five ANFIS layers

Layer	What the code does	Result
1. Fuzzification	Evaluate triangle/shoulder functions	Membership degrees
2. Rule firing	Apply min for AND and max for OR	Raw firing strengths w_i
3. Normalization	Divide by sum of active strengths	Normalized strengths w_{bar_i}
4. Consequents	Evaluate a linear function for each active rule	Crisp f_i
5. Output	Sum normalized strength times consequent	Mastery = $\sum(w_i f_i) / \sum(w_i)$

For rule i , the consequent is:

$$f_i = p_i H + q_i C + r_i P + s_i W + t_i K + b_i$$

CHAPTER 5

ANFIS training and evaluation

What is learned and what stays fixed

Component	Status	Reason
Membership shapes and breakpoints	Fixed	Easy to inspect and explain
Named premise rules	Fixed	Pedagogical meaning remains stable
First-order consequent coefficients	Trained	Fit the declared target relationship
Decision thresholds	Fixed policy	Separate from the predictive model

Synthetic bootstrap records

Because historical learner telemetry was unavailable, the training command generates 180 deterministic records using seed 42. The five profiles are strong, struggling, improving, overchallenged, and fast-incorrect. Code rows always have unknown correctness.

The synthetic label is a declared teaching heuristic:

$$\text{Target} = 0.42 \text{ performance} + 0.24 \text{ completion} + 0.14 \text{ speed} + 0.12 \text{ confidence} + 0.08 \text{ history} + \text{difficulty adjustment}$$

Gradient update

For every sample, the model predicts a weighted TSK output, computes $\text{error} = \text{prediction} - \text{target}$, and updates each active rule's coefficients in proportion to its normalized influence:

$$\text{weight} := \text{weight} - \text{learning_rate} * 2 * \text{error} * \text{rule_influence} * \text{feature}$$

The versioned artifact uses 650 epochs and learning rate 0.00002. A deterministic 20% holdout is removed before fitting.

Stored model metadata

Item	Value	Correct interpretation
Samples	180 total; 144 train; 36 holdout	A reproducible assignment-scale bootstrap
MAE	3.5227	Mean absolute error on synthetic holdout labels
RMSE	4.3674	Typical squared-error scale on synthetic holdout
R-squared	0.9364	Fit to the declared synthetic relationship
Training mode	synthetic_only	Not evidence of performance on real learners

Viva-safe wording

Say: 'The holdout demonstrates that the implemented training pipeline learns the deterministic synthetic target without fitting on its evaluation rows. It does not establish educational effectiveness for a real student population.'

Reproduction commands

```
docker compose run --rm backend python manage.py migrate --noinput
docker compose run --rm backend python manage.py seed_anfis_training_data --count 180 --seed 42 --clear-existing
docker compose run --rm backend python manage.py train_anfis --epochs 650 --validation-fraction 0.2 --split-seed 42 --include-synthetic-only
docker compose run --rm backend python manage.py evaluate_anfis --min-samples 30 --include-synthetic-only
```

CHAPTER 6

Mamdani cognitive-friction engine

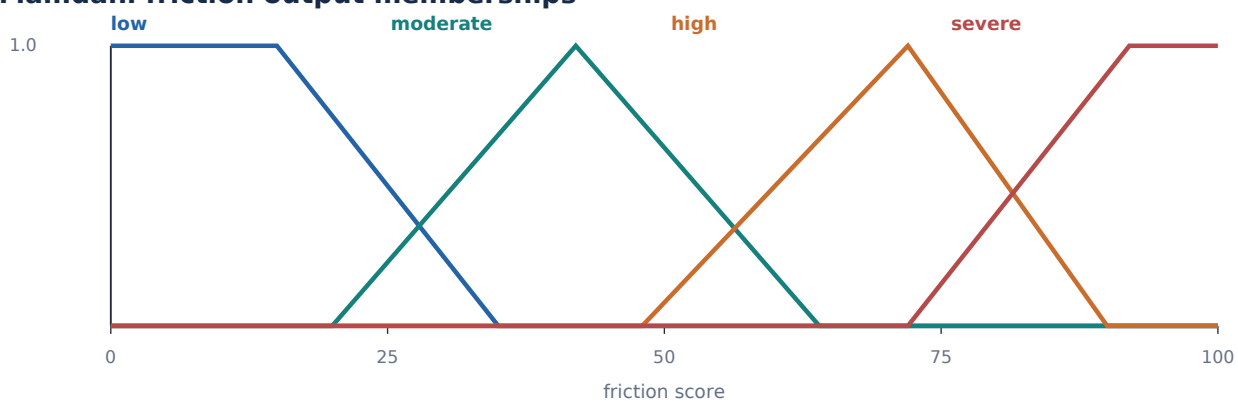
Purpose and inputs

The Mamdani system estimates interaction difficulty from observable behavior. It deliberately excludes answer correctness, because an incorrect answer is a knowledge signal, not proof of frustration.

Input	Sets	Parameters
Relative response time	Low / Normal / High	left(0.45,0.95) / tri(0.65,1.00,1.45) / right(1.15,2.10)
Assistance	Low / Medium / High	left(0,1.5) / tri(0.5,2,3) / right(2,3)
Completion	Incomplete / Partial / Complete	left(0.15,0.55) / tri(0.25,0.60,0.95) / right(0.70,1.00)

Output sets

Mamdani friction output memberships



Behavioral rules

Rule	Antecedent	Output
steady_complete	low time AND low assistance AND complete	low
normal_complete	normal time AND complete	low
minor_delay_or_hint	(normal time AND medium/high hints) OR (high time AND complete)	moderate
partial_progress	partial AND (normal time OR medium/high hints)	moderate
slow_with_support	high time AND medium/high hints	high
blocked_incomplete	(incomplete AND high time) OR (incomplete AND high hints)	severe
code_workspace_load	every code task, fixed strength 0.35	moderate

Code-load rule: every code task activates moderate friction at strength 0.35. This represents extra recall and editing load without treating code work as failure.

Mamdani inference in code

For each x from 0 to 100 in steps of 0.25, every active consequent is evaluated and clipped at its rule strength. The clipped consequents are combined with max:

$$\mu_{\text{aggregate}}(x) = \max_i(\min(w_i, \mu_{\text{consequent}_i}(x)))$$

The crisp output is the centroid of the aggregated area:

$$\text{friction} = \frac{\sum(x * \mu_{\text{aggregate}}(x))}{\sum(\mu_{\text{aggregate}}(x))}$$

The numerical method is explicit and reproducible. The trace contains memberships, active rule strengths, output-set parameters, aggregated area, and the method name `centroid_of_aggregated_output_area`.

CHAPTER 7

Complete worked inference

The following example was executed against the current backend, so the numbers match the stored trained artifact and runtime code.

Input	Value
Task weight W	55 (intermediate)
Historical mastery H	68
Completion C	1.0
Correctness	True, therefore performance P = 100
Relative time	1.35
Hints	2
Task type	MCQ

ANFIS calculation

Selected memberships:

Family	Non-zero memberships
Challenge	foundation 0.2000; intermediate 0.8696
History	medium 1.0000
Completion	high 1.0000
Performance	strong 1.0000

Only the developing-mastery rule fires. Its first branch is $\min(\text{medium history, high completion}) = \min(1,1) = 1$.

Using the trained developing-mastery coefficients:

$$f = 0.174115(68) + 0.447273(100) + 0.459491(100) - 0.043533(55) + \text{bias}$$

The raw linear value is slightly above 100, so the consequent is clamped to 100. With one active rule at strength 1, the final mastery is 100.00.

Mamdani calculation

Family	Non-zero memberships		
Time	normal 0.2222; high 0.2105		
Assistance	medium 1.0000		
Completion	complete 1.0000		
Active rule		Strength	Consequent
normal_complete		0.2222	low
minor_delay_or_hint		0.2222	moderate
slow_with_support		0.2105	high

After clipping, max aggregation, and centroid defuzzification, the friction score is 43.48. The aggregated fuzzy area is 19.2669.

Controller result

Mastery is high, but friction is not below the 35-point increase threshold. Friction is also below the 55-point reduction threshold. Therefore the controller holds the current tier.

Output	Result
Knowledge Mastery	100.00
Cognitive Friction	43.48
Focus state	Needs Support
Recommendation	hold_current_tier
Message	Continue in the current band with an optional hint or short explanation.

Interpretation: the learner demonstrated strong performance, but required time and assistance. The two-engine design avoids both overreacting downward and advancing too aggressively.

CHAPTER 8

Decision controller and adaptation

Focus-state thresholds

Condition	State
friction < 25 AND mastery >= 65	Focused & Steady
otherwise, friction < 55	Needs Support
friction >= 55	Frustrated

Difficulty recommendation

Condition	Recommendation	Target
mastery >= 75 AND friction < 35	increase_or_hold_high_tier	+1 difficulty, maximum 3
mastery < 45 OR friction >= 55	reduce_difficulty_and_show_support	-1 difficulty, minimum 1
all other combinations	hold_current_tier	same difficulty

Focus state and recommendation are intentionally different outputs. A learner can be labelled Needs Support while still advancing if mastery is high and friction is under 35.

Selecting the actual task

- Remove every task already submitted in the session.
- During the first six module attempts, prefer the underrepresented task type so the evidence includes both MCQs and code.
- Minimize absolute distance from the requested difficulty.
- Use difficulty and task ID as deterministic tie-breakers.
- If the module has ended, activate the next unfinished module and repeat selection.
- If no task remains anywhere, report curriculum completion.

Smoothing over time

Both session-wide and module-local scores use an exponential update:

$$\text{new aggregate} = 0.65(\text{previous aggregate}) + 0.35(\text{current engine score})$$

This makes the newest attempt important without allowing one unusual response to completely replace the learner model.

Module exit policy

After at least six attempts, a module is marked mastered only if all four conditions hold:

- aggregate module mastery is at least 75;
- aggregate module friction is below 40;
- three recent MCQ results are available; and
- at least two of those three MCQs are correct.

Otherwise the learner continues through the module's 15 tasks. Exhausting the bank advances the learner but records `bank_exhausted`, not false mastery.

CHAPTER 9

Explainability, persistence, and UI

What engineTrace contains

Trace area	Contents
ANFIS	Memberships, active rules, strength, crisp rule output, correctness signal, trained coefficients, model metadata, guard state
Mamdani	Memberships, active rules, strength, consequent, output-set definitions, aggregated area, defuzzification method, guard state
Controller	Threshold values used for focus and recommendation
Input snapshot	The sanitized measurements that produced the decision

Every inference is stored in FuzzyEvaluationLog. This makes a submission auditable: the system can show not just the result, but the inputs, memberships, rules, and thresholds that produced it.

Learner-facing explanation

The XAI panel shows two gauges, focus state, action, support message, and the reason for the selected difficulty. The detailed rule trace is secondary. This is a UX choice: explain enough to build trust without forcing a learner to interpret raw fuzzy mathematics.

Core persisted models

Model	Why it exists
LearnerSession	Current location, global aggregates, completed count, latest recommendation
ModuleProgress	Module-local mastery/friction, attempts, terminal status and reason
TaskSubmission	Answer, timing, completion, correctness, hint count, before-state snapshots
HintEvent	Immutable progressive hint reveals and reveal time
FuzzyEvaluationLog	Full reproducible engine input and trace
MicroSurveyResponse	Five-task satisfaction, difficulty, confidence, optional comment

Security and evidence integrity

- Public task payloads omit answer keys, answer guides, explanations, and unrevealed hints.
- The backend checks that a submitted task is the session's current task.
- Correctness is derived against the private MCQ key.
- Hint counts come from immutable server events.
- Completion is derived by validation; blank or unchanged code is rejected.
- Database writes occur inside an atomic transaction and module progress is row-locked.

CHAPTER 10

Code map and debugging route

Use this map when demonstrating the implementation. Start with orchestration, then enter each engine, then show adaptation and persistence.

Question	File / function
Where are trusted inputs derived?	backend/apps/learning/serializers.py - SubmissionSerializer.validate
Where does a submission run the engines?	backend/apps/learning/views.py - submissions
Where are shared fuzzy shapes defined?	backend/apps/fuzzy/engines/utils.py
Where does ANFIS fuzzify and fire rules?	backend/apps/fuzzy/engines/anfis.py - memberships, rule_strengths
Where is the TSK output calculated?	anfis_training.py - feature_vector, consequent_output
Where are trained coefficients stored?	backend/apps/fuzzy/trained/anfis_parameters.json
Where is Mamdani inference implemented?	backend/apps/fuzzy/engines/mamdani.py
Where are both outputs combined?	backend/apps/fuzzy/engines/controller.py
Where is the next task selected?	backend/apps/learning/adaptation.py
Where is module exit checked?	backend/apps/learning/progress.py - evaluate_module_exit
Where are scores displayed?	frontend/src/components/XAIPanel.jsx

Standalone engine demonstration

The endpoint `POST /api/fuzzy/evaluate/` runs both engines without creating a learning submission. It is useful for changing one variable at a time during a presentation.

```
{
  "taskMetricWeight": 55,
  "historicalGradeAverage": 68,
  "relativeResponseTime": 1.35,
  "assistanceInteractions": 2,
  "completionRatio": 1,
  "taskType": "mcq",
  "isCorrect": true
}
```

Recommended debugging sequence

- Inspect inputSnapshot. If it is wrong, the problem is in submission derivation, not fuzzy inference.
- Inspect memberships. Verify breakpoints and scales, especially completion [0,1] versus scores [0,100].
- Inspect active rule strengths. Recalculate one min/max expression manually.
- For ANFIS, reproduce one linear consequent and the weighted average.
- For Mamdani, confirm active consequents and aggregatedArea before checking the centroid.
- Inspect thresholds, requestedDirection, applied direction, and catalog constraints separately.

CHAPTER 11

Verification and limitations

What the automated tests establish

- ANFIS strong evidence produces more mastery than weak evidence.
- Returned ANFIS rule traces reproduce the final mastery score.
- Operational input grids always activate an ANFIS rule or its visible guard.
- Mamdani stalled/incomplete behavior produces more friction than steady completion.
- Returned Mamdani traces reproduce centroid and aggregated area.
- Public task contracts do not expose solutions or unrevealed hints.
- Clients cannot inject correctness, completion, assistance, or answer metadata.
- Task selection is non-repeating and obeys module exit and bank-exhaustion rules.
- Survey milestones persist and cannot be submitted twice.

Important limitations

Limitation	Why it matters	Defensible statement
Synthetic ANFIS data	Metrics do not prove learner effectiveness	The pipeline and holdout are demonstrated; external validity is not claimed
Code not graded	Mastery evidence is conservative	Completion is behavioral evidence; recent MCQs protect module exit
Fixed premise sets	Breakpoints are expert-designed	Transparency was prioritized; later data could calibrate premises
Binary completion	No partial runtime score in normal submissions	The fuzzy engine supports partial values, but the current UI contract derives 0 or 1
Anonymous SQLite deployment	Not production identity/storage	Appropriate for assignment demonstration; production needs auth and governance

A subtle implementation point

ANFIS consequents are clamped individually to [0,100] before the weighted average. This guarantees bounded rule outputs but can cause saturation at 0 or 100, as seen in the worked example. That behavior should be described honestly rather than hidden.

Run the verification suite

```
docker compose run --rm backend python manage.py test -v 2
docker compose run --rm backend python manage.py spectacular --validate
docker compose run --rm frontend npm test
docker compose run --rm frontend npm run build
```

CHAPTER 12

Oral-exam and presentation preparation

One-minute explanation

'The learner submits an answer, and the backend derives trusted measurements: task weight, previous module mastery, relative time, hints, completion, task type, and MCQ correctness when available. ANFIS fuzzifies performance-related evidence, fires five interpretable rules, evaluates trained first-order TSK consequents, and returns mastery. In parallel, Mamdani fuzzifies time, hints, and completion, clips and aggregates low-to-severe friction consequents, and uses centroid defuzzification. The controller increases difficulty only for high mastery and low friction, decreases it for low mastery or high friction, and otherwise holds. The selected task is the closest unused task, and the complete reasoning is stored in engineTrace.'

Likely viva questions

Why use fuzzy logic instead of hard thresholds for every input?

Overlapping linguistic sets represent gradual transitions. A response time can be partly normal and partly high, so multiple plausible rules contribute smoothly.

Why are there two engines?

Mastery and friction are different constructs. Separating them prevents wrong equals frustrated and correct equals comfortable assumptions.

What makes the first engine ANFIS-like?

It follows premise fuzzification, rule firing, normalization, first-order TSK consequent evaluation, and weighted aggregation, with trainable consequent parameters.

What exactly was trained?

Only the six consequent coefficients for each of five rules. Membership functions and premise rules remain fixed for interpretability.

How does Mamdani defuzzification work?

Active output sets are clipped at rule strength, aggregated pointwise with max, then converted to a crisp score using the centroid of the aggregated area.

Why not use correctness in the friction engine?

Correctness measures knowledge evidence. Friction is inferred from delay, hints, and incomplete behavior; mixing them would conflate two constructs.

Why does a code task add moderate friction?

Writing code has baseline interaction and recall load. A 0.35 moderate rule represents this without treating code use as failure.

What happens if no rule fires?

Both engines expose a mixed-signal coverage guard. This keeps inference fuzzy and auditable instead of silently returning an unexplained constant.

How do you avoid answer-key leakage?

The public task serializer omits private solutions; MCQ correctness is computed only on the backend.

How do you avoid a learner faking zero hints?

Assistance comes from persisted HintEvent records, not a browser field.

Why is completed code given performance 65?

It acknowledges completion but remains below correct-MCQ evidence because code correctness is unknown.

What do the synthetic holdout metrics prove?

They prove the training implementation learns its declared deterministic relationship on unseen synthetic rows. They do not prove educational validity.

Why use smoothing?

The 65/35 update reduces sensitivity to one anomalous response while remaining responsive to recent evidence.

Can a high-mastery learner receive support?

Yes. Focus and action are separate. High mastery with mild friction may advance or hold while keeping a hint available.

How is early module mastery protected?

It needs six attempts, mastery at least 75, friction below 40, and at least two correct among three recent MCQs.

What is the biggest technical limitation?

The model has not been calibrated on real learner telemetry, so membership parameters and synthetic-target assumptions require later validation.

Rapid-fire facts to memorize

Fact	Value
Modules / tasks	7 / 105; 15 per module
Difficulty weights	35 / 55 / 75
ANFIS rules	5 plus visible coverage guard
Mamdani core rules	6 plus code-load and coverage rules
Mamdani universe step	0.25 over 0-100
Increase policy	mastery $\geq$ 75 and friction $<$ 35
Decrease policy	mastery $<$ 45 or friction $\geq$ 55
Smoothing	65% previous + 35% current
Early exit	6 attempts; M $\geq$ 75; F $<$ 40; 2/3 MCQs correct
Survey interval	Every 5 completed tasks

APPENDIX A

Compact formula sheet

Fuzzy memberships

triangle: 0 outside $[a,c]$; rising $(x-a)/(b-a)$; falling $(c-x)/(c-b)$

left shoulder: 1 below a ; $(b-x)/(b-a)$ between a and b ; 0 above b

right shoulder: 0 below a ; $(x-a)/(b-a)$ between a and b ; 1 above b

Rule operations

AND = min; OR = max; Mamdani clip = min; aggregate = max

ANFIS

$$f_i = p_i H + q_i C + r_i P + s_i W + t_i K + b_i$$

$$\text{mastery} = \text{sum}(w_i f_i) / \text{sum}(w_i)$$

Mamdani

$$\mu_{\text{aggregate}}(x) = \max_i(\min(w_i, \mu_{\text{consequent}_i}(x)))$$

$$\text{friction} = \text{sum}(x \mu_{\text{aggregate}}(x)) / \text{sum}(\mu_{\text{aggregate}}(x))$$

Progress and policy

$$\text{aggregate_new} = 0.65 \text{ aggregate_old} + 0.35 \text{ score_current}$$

increase iff $M \geq 75$ and $F < 35$; decrease iff $M < 45$ or $F \geq 55$; otherwise hold

Glossary

Term	Meaning in FuzzyTutor
Fuzzification	Convert a crisp measurement into membership degrees
Premise	The IF/antecedent side of a rule
Consequent	The THEN/output side of a rule
Firing strength	How strongly an input activates a rule
TSK	A fuzzy system whose consequents are constants or functions
ANFIS	Adaptive Neuro-Fuzzy Inference System; here used to train first-order consequents
Mamdani	A fuzzy system with fuzzy output sets and explicit defuzzification
Centroid	Centre of gravity of an aggregated fuzzy output area
Coverage guard	Visible rule used when compact named rules leave a valid combination uncovered
Cognitive friction	Behavioral indication that task interaction requires effort/support
Engine trace	Stored inputs, memberships, rules, outputs, and thresholds for auditability

APPENDIX B

References and implementation sources

Implementation sources

This guide was derived from the current FuzzyTutor repository, especially the following files:

- backend/apps/fuzzy/engines/utils.py
- backend/apps/fuzzy/engines/anfis.py
- backend/apps/fuzzy/engines/anfis_training.py
- backend/apps/fuzzy/engines/mamdani.py
- backend/apps/fuzzy/engines/controller.py
- backend/apps/fuzzy/trained/anfis_parameters.json
- backend/apps/learning/serializers.py
- backend/apps/learning/views.py
- backend/apps/learning/adaptation.py
- backend/apps/learning/progress.py
- frontend/src/components/XAIPanel.jsx

Theory references

Jang, J.-S. R. (1993). ANFIS: Adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3), 665-685.

Mamdani, E. H., and Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1), 1-13.

Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353.

Ross, T. J. (2010). *Fuzzy Logic with Engineering Applications*, 3rd edition. Wiley.

End of study guide

Best final check before presenting: explain the worked example without reading, then open engineTrace and point to the memberships, active rules, and threshold decision that support your explanation.